Matteo Palmonari

matteo.palmonari@disco.unimib.it

Modeling Knowledge Graphs:

RDFS and OWL Hands-on Session

*Exercises **with Solutions***

RDFS and OWL Hands-on Session

*Ontology-based Data Modeling
*RDFS/OWL Axioms and Reasoning
*Modeling patterns (subclass-of / part-of)

Structure of the Hands-on Session

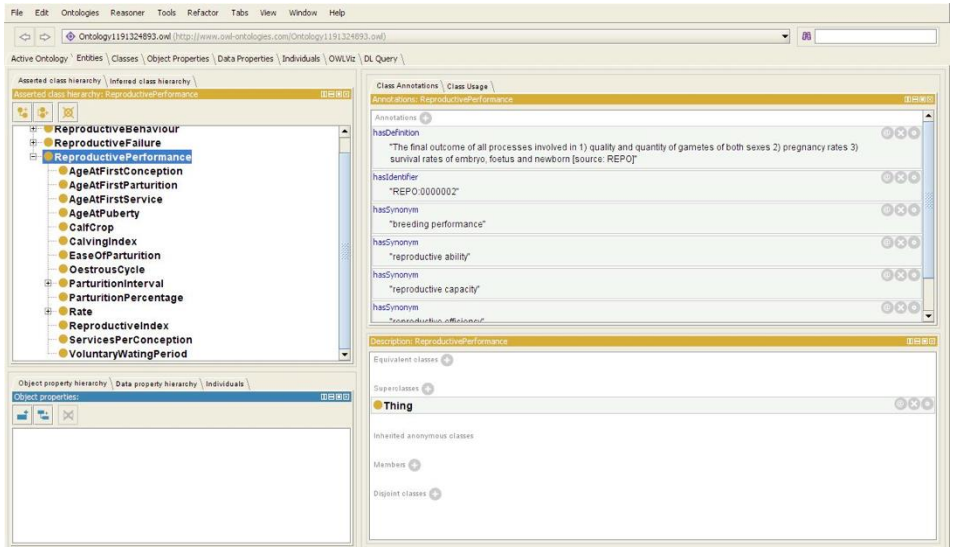
- Video introduction to Protégé
  - How to edit ontologies with Protégé (3 videos)
- Task specification without solutions
  - Levels 1-2-3 (to do); Level 4 (not mandatory)
- Task specification **+ solutions and comments**
  - Solutions are provided as .owl files that you can upload and check in Protégé + comments in the slides
  - Includes important “take-home messages” at the end
- Discussion in video-call

This presentation

Ontology Editing

State-of-the-art open source editor:
Protegè

- Graphical support for design and editing (for TBox and ABox)
  - Editing and representation with Description Logic syntax and automatic generation of OWL and RDF code
  - Basic checks
  - Visualization Tools



Integration with state of the art reasoners, rule based systems and query systems

Download Protégé at:
https://protege.stanford.edu/

Check the short video tutorials about Protégé before you start

Start to try it by opening an existing ontology, e.g., the Pizza ontology.
→ "Open from URL" using "http://www.cs.ox.ac.uk/isg/ontologies/UID/00793.owl"
Also check: https://protege.stanford.edu/support.php

Ontology Modeling: High-level Objectives

- Build an ontology with Protégé, which represents geographical entities, organizations and their connections using axioms in RDFS/OWL
  - test different modeling tasks in Protégé
- Populate the ontology with few instances
  - remember that the goal of terminological axioms (Tbox) is to model data (Abox) not to define an abstract model
- Iteratively use the reasoner to check inferences computed from the axioms
  - Get confident with reasoners and understand the consequences of the introduced axioms

Ontology Modeling: Recommendations

- You need to fulfill the specifications (specs), which are grouped into three different levels
  - the ontology should consider all the concepts / properties / assertions specified, but
  - you can extend the ontology as you want, e.g., introduce new concepts / properties to better organize the ontology
- Use always the same naming startegy:
  - CamelCaseClass; camelCaseProperty
  - Underscore_Class; underscore_Property
- Read all the specs for each level before starting to edit the ontology, but it could be helpful to check solutions after each level
- **Remark:** I won't provide an image (e.g., a graph) as input, because this would be part of the solution and it could be in fact be an intermediate step of your exercise

Ontology Modeling: Level 1 – Specs 1

- Introduce the following classes:
  - Continent, Country, Region, City
  - Organization, Company, NonProfitOrganization [=NGO]
- For Country, define subclasses on a geographical basis (eg EuropeanCountry),
  - to support grouping of countries based on the continent in which they appear
- Define entities for each of the classes (Continent, Country, Region, City) and (Company, NoProfitOrganization), as suggested below:
  - Continents (all); Countries (Italy, Germany, France, Netherlands); Region (Lombardy, Lazio, Piedmont); Cities (Milan, Roma, Berlin, Bonn, Paris); Company (FIAT, SAP), NGO (Emergency, MediciSenzaFrontiere)
  - **Remark**: to fulfill specs 3 about reasoning, where you will be asked to infer types for some of the entities, it is a good idea to specify the most specific types for some of these entities and underspecify types for some others; as a suggestion, keep Italian entities to test reasoning, so underspecify their types when inserting them (e.g., make them instances of owl:Thing).

Ontology Modeling: Level 1 – Specs 2

- Introduce properties to model the relationships between individuals of their respective classes (in the Tbox and in the Abox):
  - A property to link the **contained** geographical entities to those that contain them
  - A specific property to link cities to the countries they are **capital of**
  - A property to specify the **population** of countries (based on Wikipedia)
  - A property “hasHeadquartersIn” to specify the country where an organization has its headquarters
  - A property “hasOfficesIn” to specify where an organization has offices, including headquarters, but not limited to headquarters
- Introduce at least two Abox assertions for each property
  - For region, I suggest using Italian regions: Lombardy, Lazio
- Introduce at least two annotations using rdfs:label
  - Suggestion: use MSF for Medici Senza Frontiere and add “Medici Senza Frontiere” as a label; use NGO for NonProfitOrganization and add the full name as rdfs:label

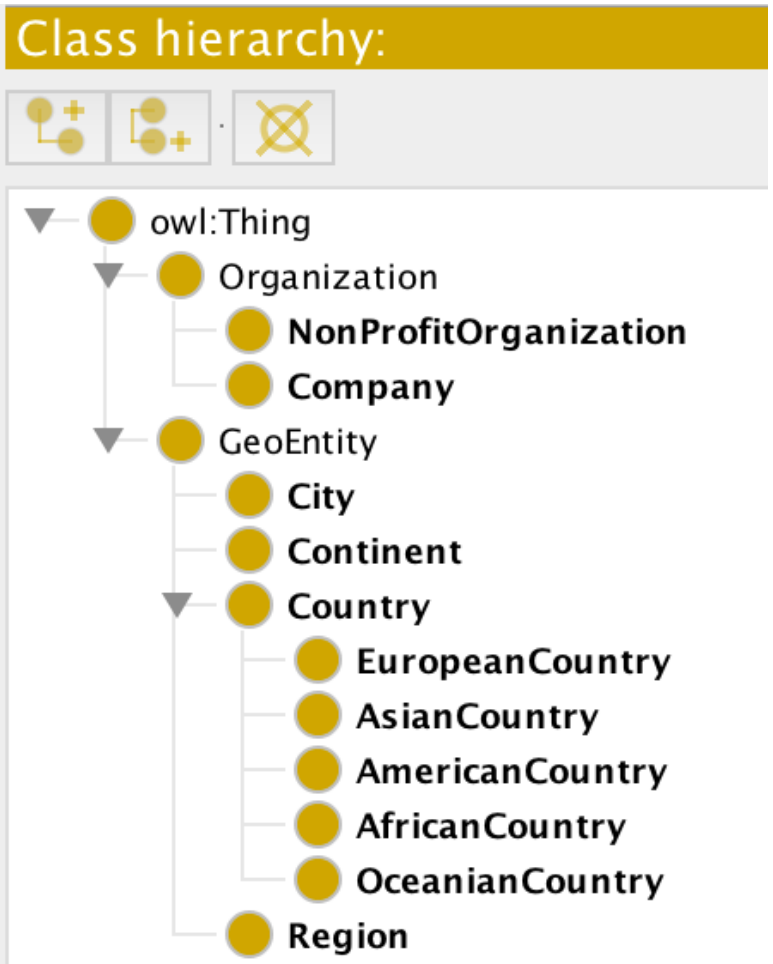
Ontology Modeling: Level 1 – Specs 3

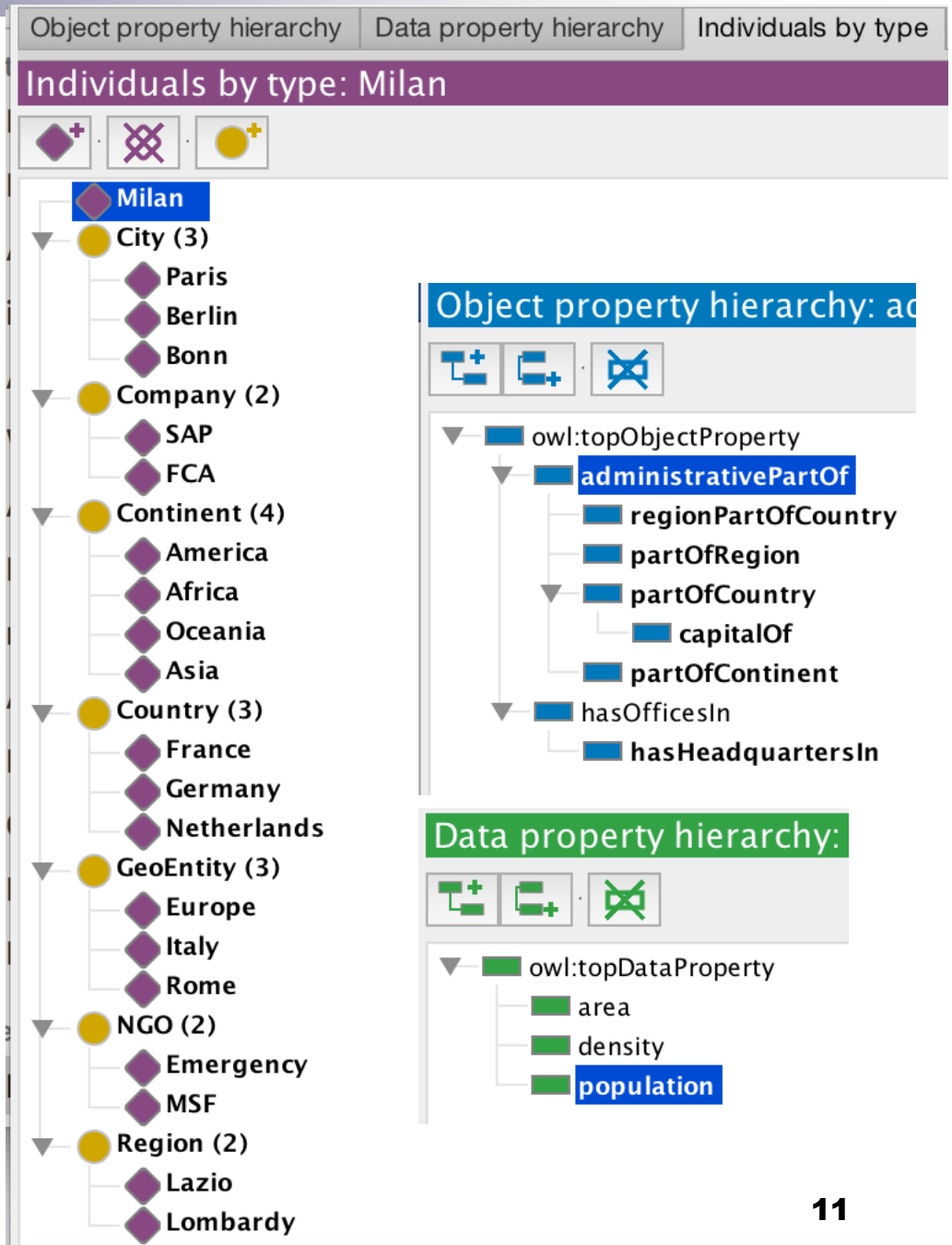
- Specify the meaning of the properties using only RDFS constructs (domain/range restrictions)
- Use a reasoner (e.g., Hermit) and calculate the inferences making sure that:
  - The Abox is defined in such a way that you can infer at least one-two classes for the instances, which were not explicitly defined (e.g., infer the type City for some city)
  - Suggestion (see also Specs 1): test type inference on Italian entities;

Solutions – Level 1

- Suboptimal solution in companies-geo.rdfs-0.9.owl
  - Here we entered type-specific relations (e.g., partOfContenent between countries and continents); this is more similar to database-style modeling, where we introduce a different relation between every type pair for which we want a relation; yet, this solution supports specific type inference using domain / range restrictions (see e.g., Italian cities)
- A better solution is in companies-geo.rdfs-1.owl
  - here a GeoEntity class is added so that we could add a property administrativePartOf as super-property of all type-specific properties;
  - we still kept all the other type-specific properties to preserve the inferential power, but in an ontology we may also decide to discard these properties to adopt a more conceptual approach to modeling (intuitively, they represent one unique relation); however, in this case, we would not be able to infer specific entity types using domain and range restrictions (we could only infer the GeoEntity type), so these relations are preserved.

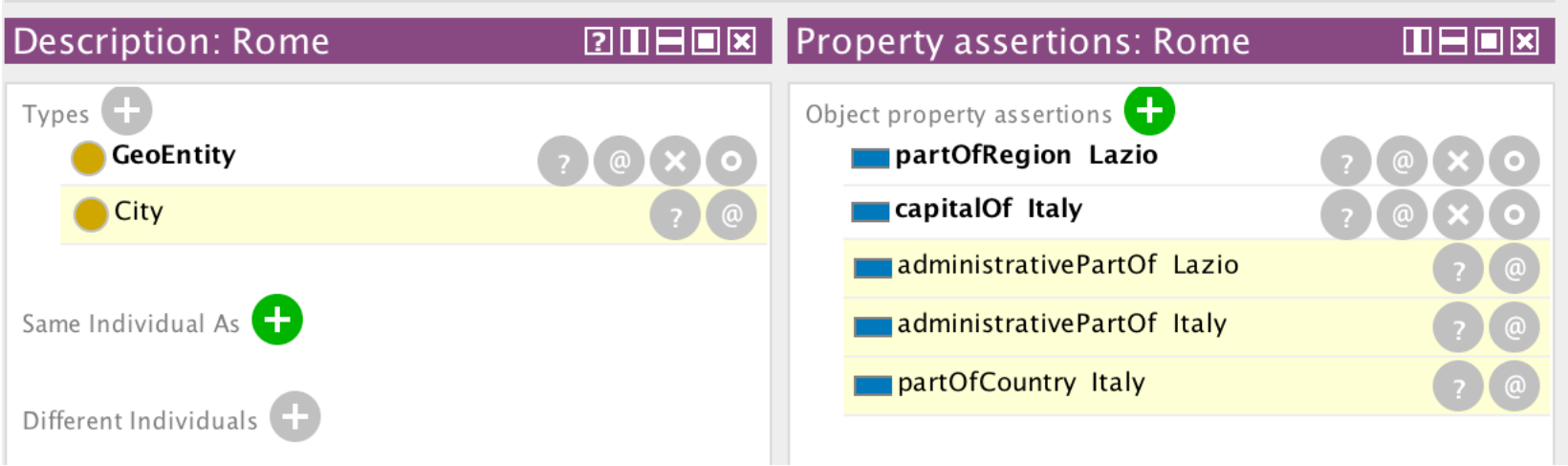
"Better" solution





Solutions: Remarks about Inferences

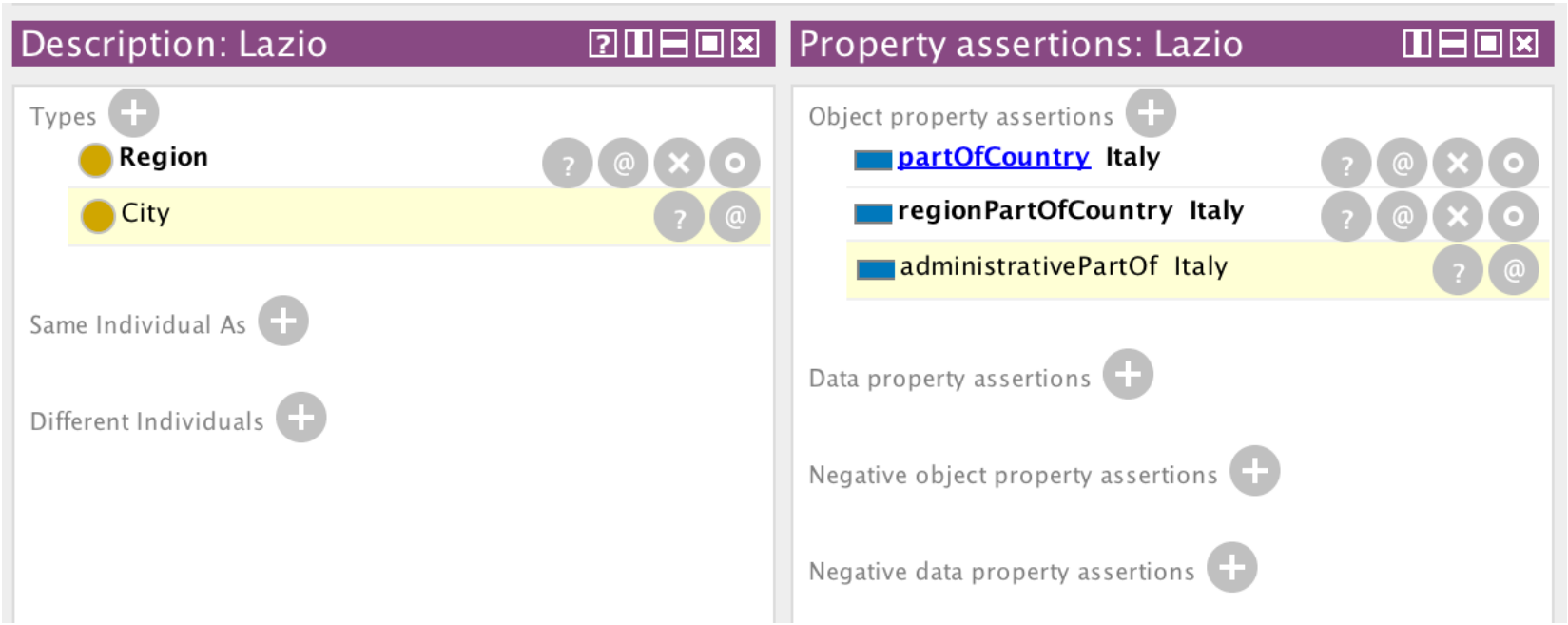
- See inferred triples because of subproperty relations (we don't need to specify that a capital is also part of a country)



Solutions: Remarks about Inferences

- Try unintended inferences (pic below) on the solution ontology
  - Enter the following triple (misuse of the partOfCountry property)

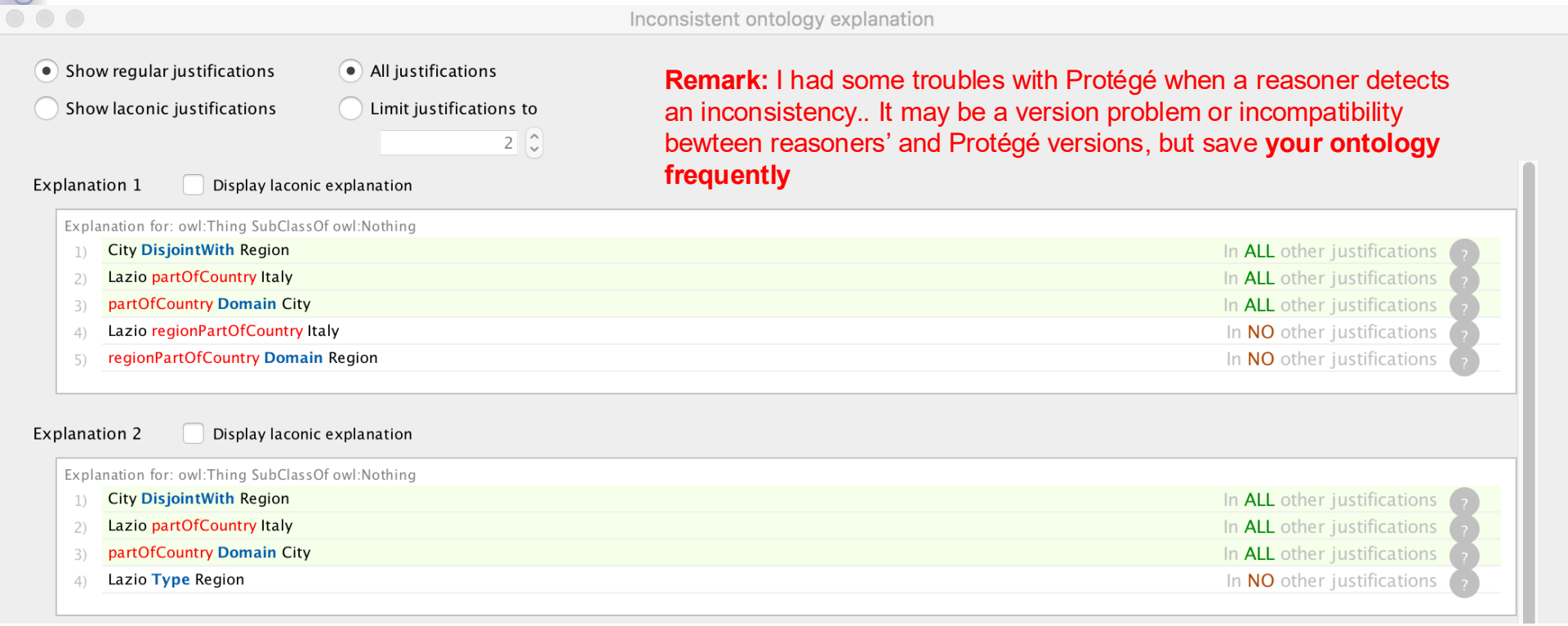
    Lazio partOfCountry Italy .
  - Not inconsistent because no axiom to state Region is disjoint from City



Solutions: Remarks about Inferences

- Try unintended inferences (pic below) on the solution ontology
  - Enter the following triple (misuse of the partOfCountry property)

    Lazio partOfCountry Italy .
  - Not inconsistent because no axiom to state Region is disjoint from City
    - If we add this axiom …



Remark: I had some troubles with Protégé when a reasoner detects an inconsistency.. It may be a version problem or incompatibility bewteen reasoners' and Protégé versions, but save **your ontology frequently**

Ontology Modeling: Level 2

- Specify disjointness between classes
- Specify new properties (one or more, generic, e.g., contains, or type-specific) as inverse of the property used to model part-of relations
  - Suggestions: do not specify domain / range restrictions for this one, see inferred domain / range restrictions using the reasoner; check Abox inferences after reasoning.
- Specify which properties are functional
- Specify which properties are transitive
- **Remarks:**
  - Run the reasoner at each step and check inferences (otherwise it will be hard to debug)
  - Pay attention to the interaction between class disjointness and domain / range restrictions to avoid inconsistencies
  - Pay attention to the interaction between axioms that specify functional and transitive properties, and, also subproperties

Solutions – Level 2 – Option 1

- One solution in companies-geo.rdfs-owl-L2-1.owl
  - Check disjointness in the file
  - **hasHeadquartersIn** is functional
  - All the type-specific properties are functional
  - **administrativePartOf** is transitive but not functional to avoid inconsistencies (transitive and functional do not mix well *); but, even if we'd remove transitivity we would still get inconsistencies for the following reasons**:
    - it has two subproperties: regionPartOfCountry (between regions and countries) and partOfCountry (between cities and countries); in fact Milan would be inferred as having two values of administrativePartOf: Lombardy and Italy
  - **administrativePartOf** is inverseOf **administrativelyContains**

Solutions – Level 2 – Option 2

- A second solution is in companies-geo.rdfs-owl-L2-2.owl, with the following differences
  - **administrativePartOf** is not transitive but functional (it represents the generic tree-like relation); to do so, we also had to remove condition **, i.e., we removed the property partOfCountry (between cities and countries) from the subproperties of **administrativePartOf**
  - **transitiveAdminPartOf** is the transitive superproperty of **administrativePartOf;** this implements the modeling pattern discussed during the lesson, where a transitive super-property provides desired inferences yet keeping the more specific relations functional to model the tree-like structure
  - **transitiveAdminPartOf** is inverseOf **administrativelyContains** to keep the latter a transitive property
- This solution contains a good modeling pattern;

Ontology Modeling: Level 3 - Specs

1. Introduce an entity that represents "Rome the Capital"
   1. specify its major (a datatype property is fine if you want to avoid adding also persons to the ontology), and state that this is the same entity as Rome and control that for each of the two entities you infer the same statements.
2. Define the range of **hasHeadquartersIn** as to be either a City or a Country (using the "class expression editor in Protégé)
   - Under the not-too-realistic - but somehow supported by the information I found in Wikipedia – assumption that headquarters are either specified as the city where they are, e.g., Paris for msf, or as a country, e.g., Netherland for FCA
3. *Define* the following classes (using the Protégé's "object restriction creator" when adding the equivalent class):
   1. Capital, as the class equivalent to the class of cities that are capitals of at least one Country.
   2. EuropeanCountry, as the class equivalent to *the class of countries that are part of Europe**; to represent *the latter class*, you need to use the following construct (see the DL and Protegè syntax), which we have only quickly mentioned in the lesson:

$\exists P.\{ e \}$ *with e being an individual*	P value *e* *with e being an individual*

   1. EuropeanCapital, as as the class equivalent to the class of capitals that are capitals of at least one EuropeanCountry (requires 2 to be specified first)

Remarks: required exercise ends with 3.1 ; 3.2, and 3.3 are for the bravest

Ontology Modeling: Level 3 - Solutions

- See the solution in companies-geo.rdfs-owl-L3-2.owl (because it extends the ontology companies-geo.rdfs-owl-L2-2.owl solution) for tasks 1 and 2
- See here alternative solutions for class definition using equivalence in DL (left) and Protégé syntax (right):

Parenthesis not needed here, added only for readability

3.1	**Capital** ≡ ∃*capitalOf*.Country	Capital ≡ City and (capitalOf some Country)
3.2	**EuropeanCountry** ≡ Country ⊓ ∃*partOfContinent*.{Europe}	EuropeanCountry ≡ Country and (partOfContinent value Europe)
3.3 (file)	**EuropeanCapital** ≡ Capital ⊓ ∃*capitalOf*.EuropeanCountry	EuropeanCapital ≡ Capital and (capitalOf some EuropeanCountry)
3.3 (ok)	EuropeanCapital ≡ Capital ⊓ ∃transAdminPartOf.EuropeanCountry	EuropeanCapital ≡ Capital and (transAdminPartOf some EuropeanCountry)
3.3 (ok)	EuropeanCapital ≡ Capital ⊓ ∃transAdminPartOf.{Europe}	EuropeanCapital ≡ Capital and (transAdminPartOf value Europe)

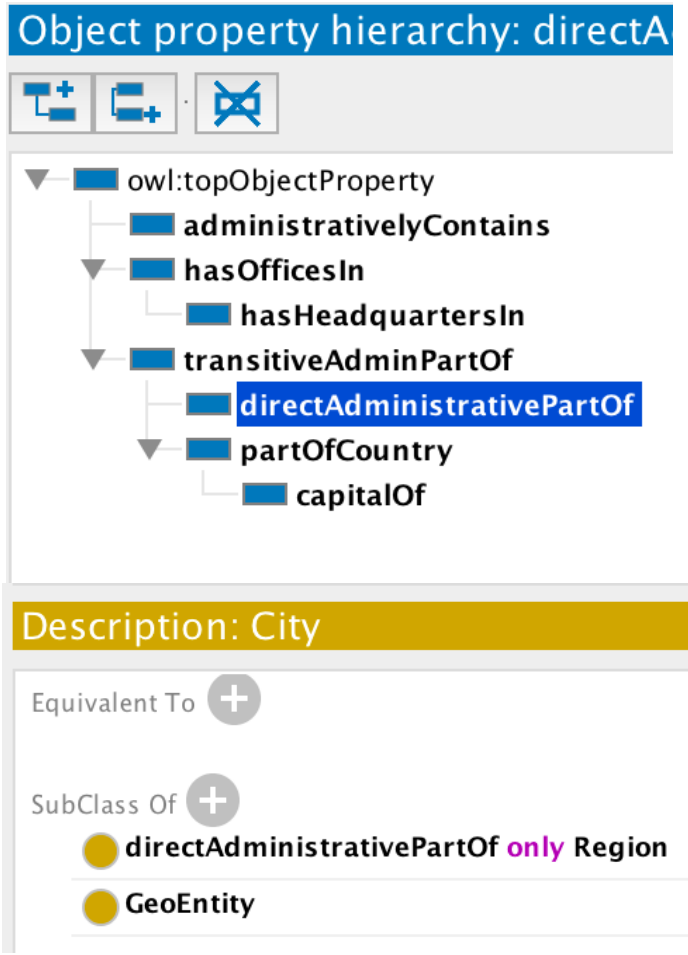
Ontology Modeling: Level 4 – Specs (for the bravest ☺)

- Try to substitute each of the type specific properties that links cities to regions, regions to countries and countries to continents with one unique non transitive but functional part of property; then specify that for this property, the range must be: regions, if the subjects are cities; countries, if the subjects are regions; continents, if the subjects are countries.
- To do so, use localized domain / range restrictions instead of global (i.e., rdfs) domain and range restrictions, i.e., (slide 42 of the presentation about OWL):

$C \sqsubseteq \forall P.D$ $C \sqsubseteq P \text{ only } D$

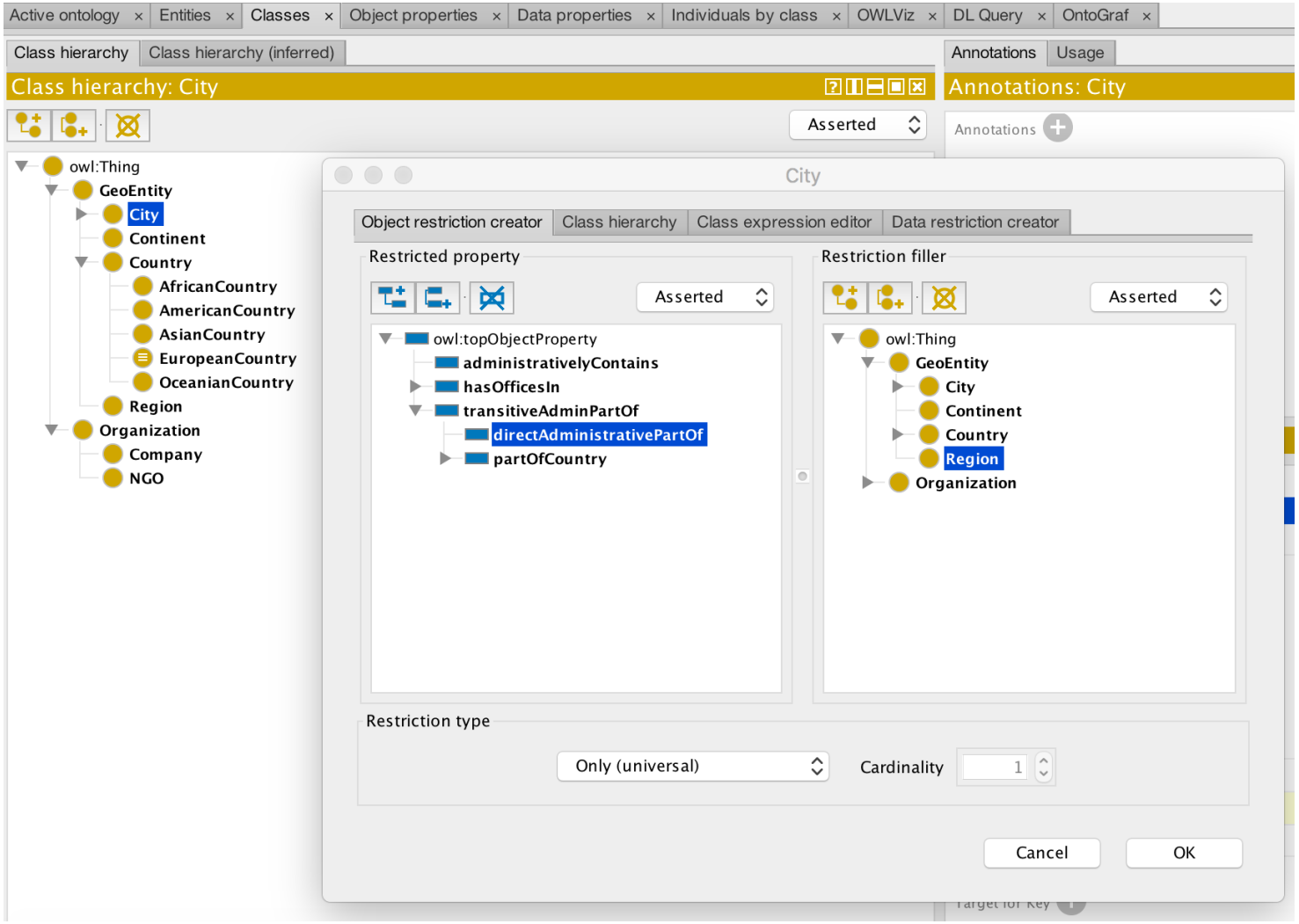
Ontology Modeling: Level 4 – Solutions (1)

- The solution is in companies-geo.rdfs-owl-L3-2.owl; few remarks:
  - A (functional) direct part of relation subclass
- Arguably the best ontology from a conceptual point of view*
  - All type specific properties are removed
  - We are now left with a (functional) direct part-of relation and its transitive super-property
  - We could also remove **partOfCountry**, use **transitiveAdminPartOf** to add explicit links between cities and countries, and place **capitalOf** as subproperty of **transitiveAdminPartOf**
- Observe that to specify localised range restrictions, we have to move from a "property view" to a "class view"
  - to specify that the values of **directAdministrativePartOf** must be regions if the subjects are cities, we have added a subclass of constraint for City (see next slide for how to specify this in Protege)



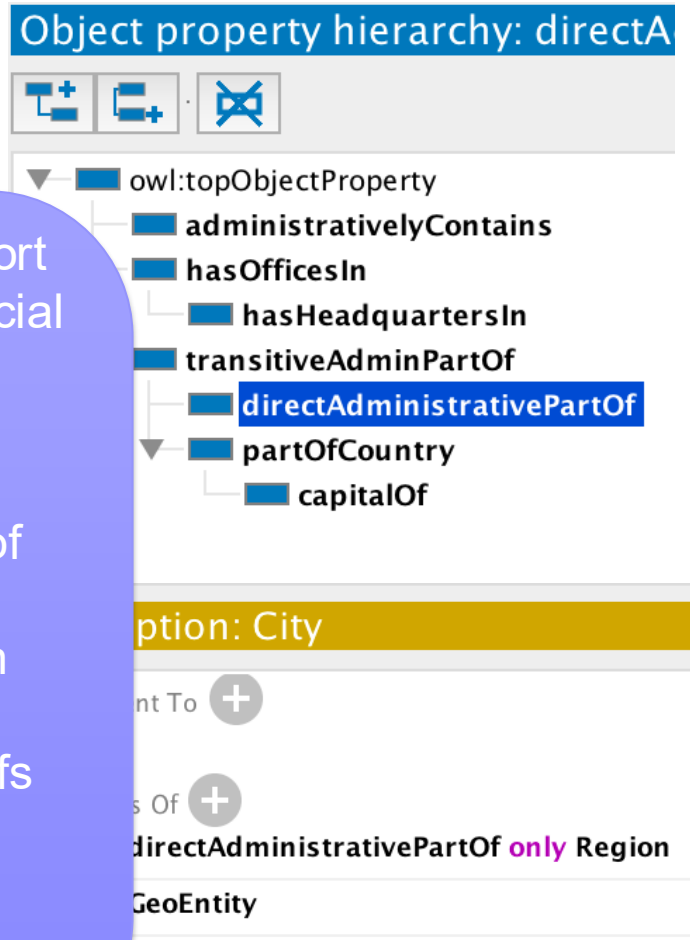
Ontology Modeling: Level 4 – Solutions (2)

How to specify localised range restrictions in Protégé



Ontology Modeling: Level 4 – Solutions (1)

- The solution is in companies-geo.rdfs-owl-L3-2.owl; few remarks:
  - A (functional) direct part of relation subclass
- Arguably the best ontology from a conceptual point of view*



*Remember that the goal of the ontology is to support
1. interoperability through a shared vocabulary (social semantics)
2. effective knowledge processing

Thus, the optimal solution from a conceptual point of view may not be the optimal solution for a given application domains. The optimal solution in a given application domain should consider also:
- Expressivity vs. efficiency (in reasoning) trade-offs
- Risk to end up with undesired inferences or inconsistencies (more constraints → more inferences → more risks of inconsistent or undesired inferences)

Latest Minor Tasks

- Upload the DBpedia ontology in Protégé and check it
  - You will see the solutions adopted to consider the trade-offs discussed in previous slide: much underspecified ontology to avoid inconsistencies
- Export the HTML documentation automatically generated by Protégé from your ontology and check it
  - Tools –> Export OWLDoc

Hands-on Session Take-home Message (1)

What we expect you have learned…

- Language expressivity and graph data modeling with declarative semantics
  - RDFS on top of OWL basic distinctions (object vs. datatype properties, class / property lattices):
    - Level 1: proficient
  - Basic OWL & Classification:
    - Level 2: good
    - Level 3, Tasks: 1 [sameAs], 2 [boolean constructors], 3.1 [basic classification]): good; Tasks 3.2 and 3.3 [advanced classification]: moderate
  - Advanced OWL [localised range restrictions]:
    - not mandatory
- Covered modeling patterns:
  - Interaction between taxonomic representation and part-of like relations
    - very important and very common modeling task; e.g., part-of based taxonomies are not modeled using subclass of relations because they encode different semantics (common mistake when newbies with ontology-based modeling)
  - Fundamental role of property characteristics in formal specification of meaning (no "real semantics" beyond what is explicitly constrained)

Hands-on Session Take-home Message (2)

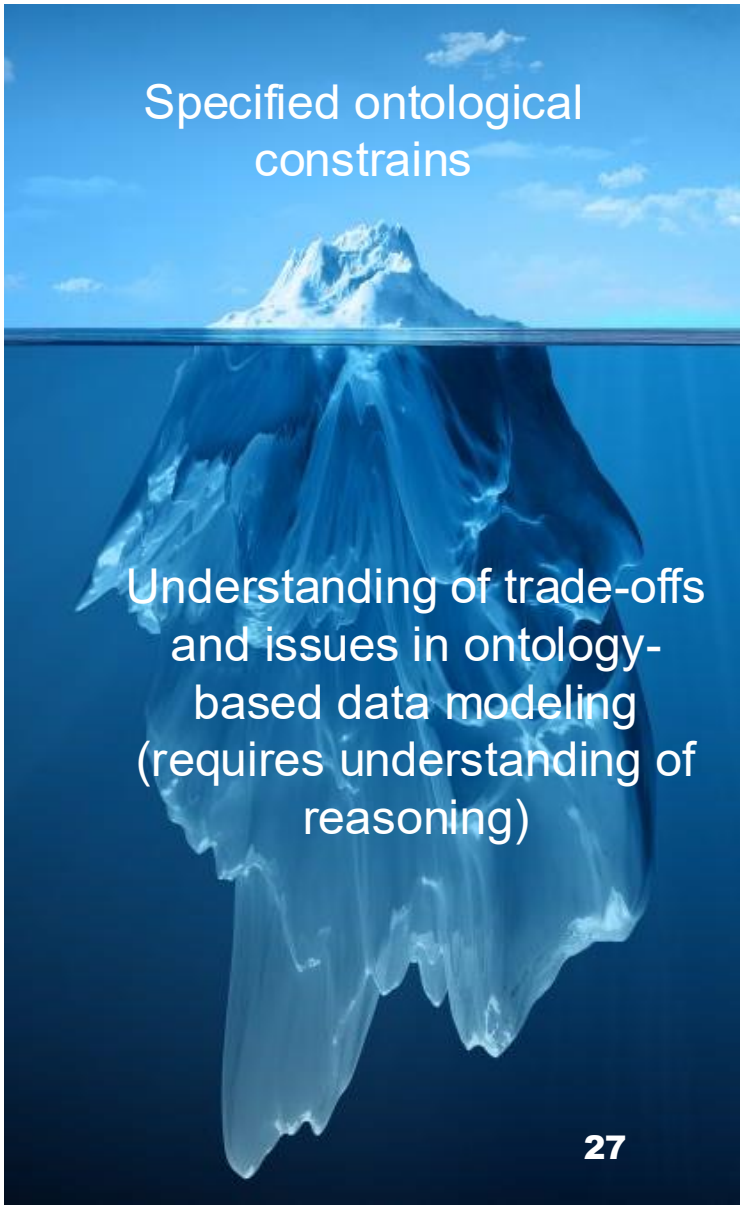
Some latest remarks (1) …

- Many different options to model even a simple domain
  - In applications we should consider several trade-offs:
    - Expressivity vs. efficiency
    - Inferential power vs. risks of inferring undesired facts
      - esp. when data are extracted from legacy uncontrolled sources, e.g., DBpedia from Wikipedia Infoboxes)

Hands-on Session Take-home Message (3)

Some latest remarks (2) …

- "Little semantics goes a long way"
  - Sometimes we prefer to keep the ontology lightweight…
  - … but expertise on deep semantic modeling is very useful to:
    - Take more informed and wiser decisions when modeling, e.g., the Tbox must be consistent
    - Understand what you may want to replicate with work-around solutions when representing knowledge graphs with non-RDF data management frameworks;
      - property-graphs have rigid typing and no inference, but you may want to replicate basic reasoning tasks with had-hoc coding; e.g., transitive closure of relations

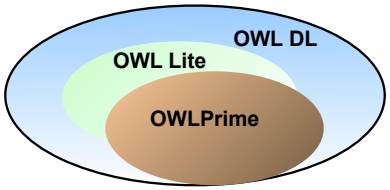


Commercial Reasoners: Oracle

OWL Subsets Supported

Most of these constructs tested in the exercises

- RDFS++
  - RDFS plus owl:sameAs and owl:InverseFunctionalProperty
- OWLSIF (OWL with IF semantics)
  - Based on Dr. Horst's pD* vocabulary[1]
- OWLPrime
  - rdfs:subClassOf, subPropertyOf, domain, range
  - owl:TransitiveProperty, SymmetricProperty, FunctionalProperty, InverseFunctionalProperty, inverseOf
  - owl:sameAs, differentFrom
  - owl:disjointWith, complementOf,
  - owl:hasValue, allValuesFrom, someValuesFrom
  - owl:equivalentClass, equivalentProperty
- **Jointly determined with domain experts, customers and partners**



https://www.oracle.com/technetwork/database/options/semantic-tech/oow08-rdfs-owl-apps-131263.pdf